

양식 등록 마법사 설계

구현 없음. 담당자가 빈 설문 한 장으로 템플릿을 만들고, 채점은 지금 엔진을 그대로 쓴다. 대상 사용자는 주민이 아니라 동 직원-운영자 한 명.

판단

상용 OMR SDK, 클라우드 문서 AI, OpenCV.js, 새 프론트 캔버스 라이브러리는 넣지 않는다. 이미 있는 Next.js + Konva + Python OpenCV + Supabase templates 위에 엔드포인트 하나와 마법사 화면 하나만 얹는다.

- 신규 API: 1 (detect_circles)
- 신규 화면: 1 (4단계 마법사)
- AI / 외부 인식 API: 0

범용성의 병목은 채점이 아니라 양식 정의다. 채점 파이프라인은 이미 템플릿 상대좌표를 받아 워프한 뒤 원 안 흑색만 켜다. 양식이 바뀌어도 그 계약만 지키면 된다. 운영자가 상자-제목-최소/최대를 넣고, 상자 안 원 후보는 OpenCV가 제안하면 AI 없이 유효한 템플릿 JSON이 나온다.

신사2동 기본 양식은 삭제하지 않는다. 내장 시드이자 회귀 기준이다. 마법사는 두 번째 양식부터를 등록하는 길이다.

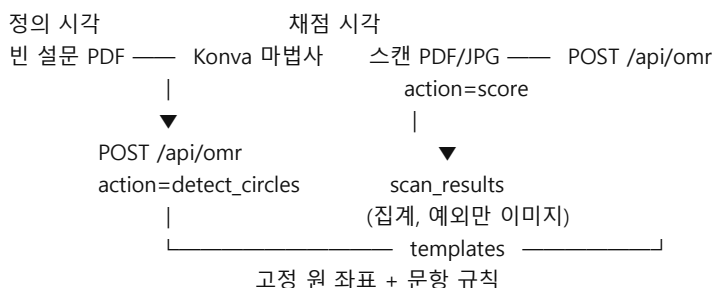
세 층

층	누가	언제	산출
양식 정의	운영자 + 검출 API	빈 설문 1장, 한 번	templates 행 (원 좌표 + 규칙)
채점	기존 /api/omr	스캔 N장	scan_results (집계·예외)
예외 수작업	기존 예외 탭	문제 장만	규칙 밖 표의 수동 반영

원을 스캔마다 다시 찾지 않는다. 검출은 빈 양식에서만 한다. 이 분리가 일반10 오탐 같은 재탐지 실패를 막는 핵심이다.

아키텍처

로컬은 Next 개발 서버가 Python OMR로 프록시한다. 프로덕션은 Vercel Python serverless 한 함수에 채점과 검출을 같이 둔다. Hobby 용량 한도 때문에 OpenCV를 두 번 묵지 않는다.



모듈 책임

- 브라우저 · 채점 경로 (유지): ScanPanel, useOmrQueue, pdfjs 페이지 분할, 예외 탭, 집계. 템플릿만 DB에서 읽어 /api/omr에 넘긴다. 원 재탐지 없음.
- 브라우저 · 양식 마법사 (확장): 기존 TemplateEditor(Konva)를 4단계 마법사로 재구성. 상자 그리기, 원 드래그, 문항 규칙 폼. Fabric/tldraw 추가 금지.
- api/omr.py detect (신규 1): 빈 양식 이미지 + 문항 ROI → 원 후보. Hough + 원형도 + 격자 스냅. 채점 함수와 파일을 공유해 opencv-python-headless를 한 번만 쓴다.
- Supabase (유지): templates 스키마가 이미 계약. markers, questions[].options.circle, min_select, max_select.

데이터 계약

마법사 산출물은 지금 TemplatePayload와 동일해야 한다. 채점이 보는 것은 원 ROI와 규칙뿐이다.

필드	출처	채점 사용
markers tl/tr/br/bl	운영자가 빈 양식에서 네 모서리	워프
questions[].label	운영자 입력	집계 헤더
min_select / max_select	운영자 입력	예외 규칙
options[].circle	검출 후보 + 사람 드래그	흑색 비율
question_region (선택)	운영자가 친 상자	아니오 · 편집용
fill_threshold	기본 0.28, 미리보기에서 조정	칠함 판정

외부 도입 심사

효율이 나오는 곳만 쓰고, 제품을 대체하거나 AI 제약을 깨는 것은 제외한다. 문서-플러그인은 구현 참고용이지 런타임 의존이 아니다.

후보	결정
OpenCV Python (기존)	채점+검출 공용
Konva / react-konva (기존)	마법사 유지
pdfjs-dist (기존)	유지
Supabase templates (기존)	유지
Azure Form Recognizer / Google Doc AI / Textract	사용 안 함
OpenAI Vision 등 LLM	사용 안 함
Remark / Gravic / SDAPS	사용 안 함
opencv.js	사용 안 함
tldraw / Fabric.js	사용 안 함
ArUco / QR 기준점	v1 제외, 나중에 선택

참고 문서만 (의존성 아님): OpenCV HoughCircles, findContours 계층(링+구멍), 원형도. 스캔에서 원을 다시 찾지 말 것. 프론트는 Konva Transformer. 새 npm 패키지는 기본적으로 추가하지 않는다.

운영자 4단계

한 화면 포토샵이 아니라 위에서 아래로 끝나는 마법사. 뒤로 가기 가능. 저장 전에는 스캔 채점에 안 쓰인다.

1. 빈 양식 — PDF/JPG 업로드, 양식 이름. 여러 장이면 1페이지만 템플릿.
2. 모서리 — 표의 좌상·우상·우하·좌하. 4개가 아니면 다음 단계 잠금.
3. 문항 상자 — 문항마다 기표 열을 드래그. 제목, 최소, 최대, 하나만/여러 개. 상자 안에서만 원 검출. 위→아래 정렬. 라벨 기본값 1, 2, 3.... 오탐은 원만 드래그/삭제/추가.
4. 확인 — 원 오버레이 검토, 채움 임계값, 저장.

친화성의 핵심은 캔버스 기능 수가 아니다. 한 번에 한 문항만 상자를 친다. 원 좌표는 기본 숨기고, 틀렸을 때만 핸들을 연다.

신사2동 예시 조작량: 모서리 4번 + 상자 3번(특화·일반·시설) + 제목 3개 + 한도 3줄. 원 15개는 검출이 맞으면 손을 안 댄다.

API 계약

같은 Vercel Python 함수, JSON action만 나눈다. 로컬 scripts/dev_omr_server.py 프록시 유지. 타임아웃 10초, 본문 4.5MB.

- action: score (기존) — 입력: image_base64, template. 출력: answers, fill_ratio, sheet_status. 워프 후 저장된 원만 채점.
- action: detect_circles (신규) — 입력: image_base64, region(x,y,w,h 상대좌표). 출력: circles[], rejected_count. 각 원: x,y,w,h, score.

검출 알고리즘: ROI 자르기 → 적응형 이진화 → 구멍이 있는 링 컨투어 → 원형도·반지름 필터 → y정렬 후 격자 스냅 → 상대좌표. Hough는 보조 후보. 입력은 빈 인쇄본이다.

마법사만 detect를 부른다. 스캔 큐는 score만 부른다. 검출 실패해도 상자는 남기고 원을 수동 추가로 진행한다.

저장은 기존 templates insert/update. 스캔 화면은 양식이 여러 개면 고르고, 하나면 기본 양식 자동 선택.

v1에 넣지 말 것

- 문항 제목 OCR
- 스캔 중 원 재탐지
- 양식 디자이너 전체 (정렬·레이어·공동편집)
- 서술·숫자 그리드 등 원-흑색 이외 유형
- 양식 PDF 자동 다페이지 템플릿 (다페이지는 스캔 쪽만)
- 임계값 자동 학습

리스크

- 검출이 표 선을 원으로 봄 → ROI를 기표 열만 작게, 링(구멍) 조건 강화
- 복사 흐린 빈 양식 → 수동 원 추가, 가능하면 원본 PDF
- 마법사 UI 공수 → 단계 4개 고정, Konva 재사용, 새 라이브러리 금지
- 양식 여러 개 혼선 → 스캔 전 양식 이름 명시, 신사2동 시드 이름 고정

이 설계의 사용자는 개발자가 아니라 양식을 등록하는 담당자다. 그래도 빈 설문 이미지와 문항 구조를 알고 있어야 한다. 주민 셀프 등록까지 가면 화면이 다시 커지므로 v1 목표가 아니다.